# Revealing Hidden Decision Patterns in Machine Learning Models

## System Analysis & Design Document

## Table of Contents


**Table of Contents** 2
**1. Problem Statement & Objectives** 3
Problem Statement 3
Objectives 3
**2. Use Case Diagram & Descriptions** 4
**3. Functional Requirements** 5
**4. Non-Functional Requirements** 6
**5. Software Architecture** 8
**5.1 Architecture Diagram** 8
**6. Data Persistence Strategy** 9
**6.1 Why No Database (or: Database Rationale)** 9
**6.2 Data Storage Schema** 10
**7. Data Flow Diagrams** 11
**7.1 Context-Level DFD (Level 0)** 11
**7.2 Level 1 DFD** 12
**8. Sequence Diagrams** 13
**8.1 Sequence Diagram — UC1** 13
**8.2 Sequence Diagram — UC4** 13
**8.3 Sequence Diagram — UC5** 14
**9. Activity Diagram** 15
**10. State Diagram** 16
**11. Class Diagram** 17
**12. Component Diagram** 18
**13. Deployment Diagram** 19
**14. UI/UX Design** 20
**14.1 Wireframes** 20
**14.2 UI/UX Guidelines** 21
14.3 Figma 22
**15. Technology Stack** 23
**16. Deployment Strategy** 23
**17. Internal API Documentation** 25
**18. Testing & Validation Plan** 30

## 1. Problem Statement & Objectives

This project analyses and interprets the decision-making behaviour of self-supervised representation learning models in artificial intelligence. The central question is whether these models—specifically CLIP evaluated in a zero-shot classification setting—truly learn meaningful semantic patterns from data, or whether they exploit irrelevant features that accidentally correlate with the correct output .
Using Explainable AI (XAI) techniques (Grad-CAM and, optionally, Layer-wise Relevance Propagation), the project visualises model attention, quantifies spurious feature reliance, and presents findings through an interactive dashboard.

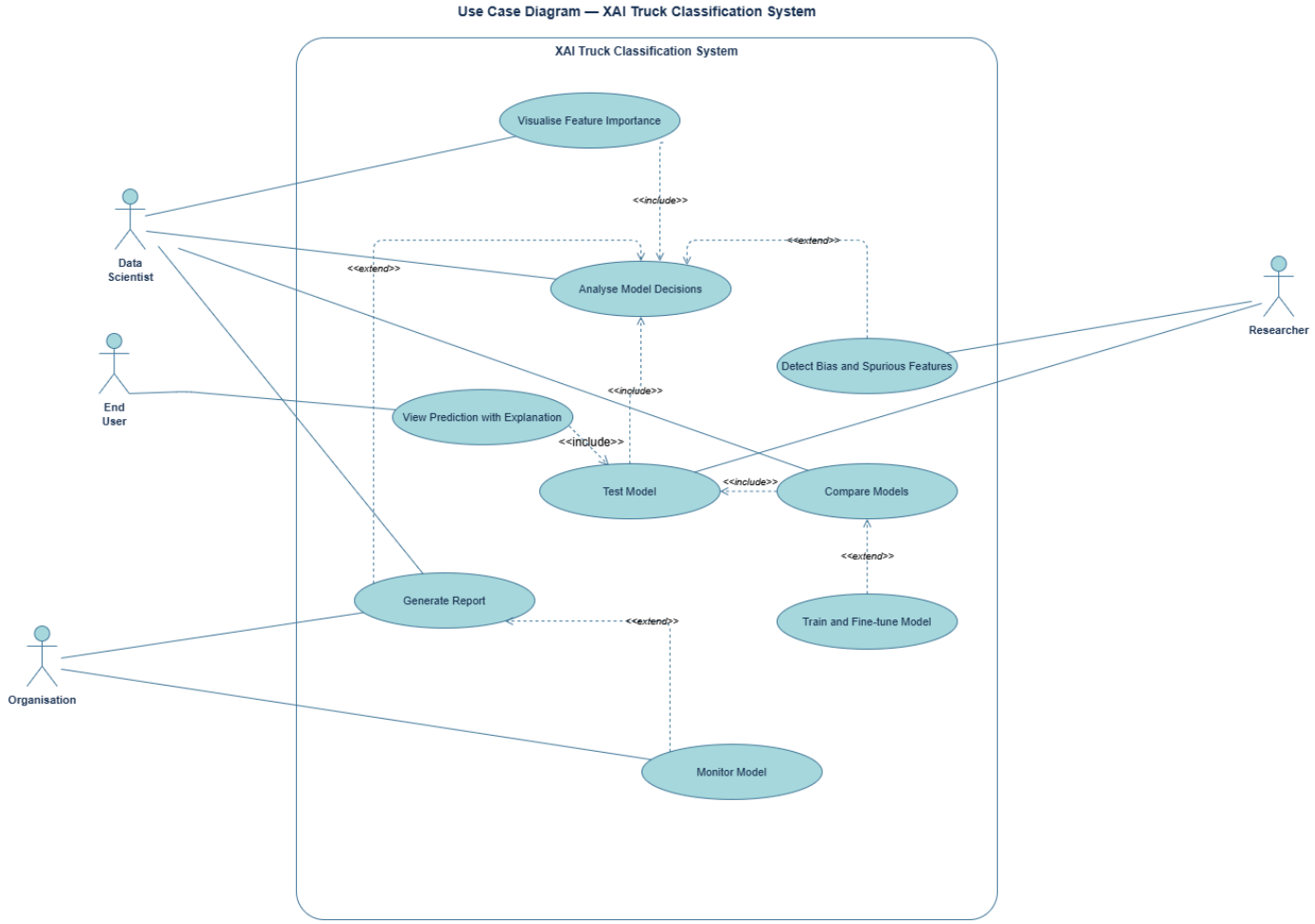
### Problem Statement

Modern AI systems rely heavily on self-supervised and contrastive learning to extract features from large, unlabelled datasets. Impressive benchmark accuracy can mask a critical flaw: models may learn dataset shortcuts rather than the genuine concept behind the task. When deployed in high-stakes applications—medical diagnosis, autonomous vehicles, industrial inspection—this shortcut learning can produce dangerous failures.
This project provides a principled, reproducible investigation of shortcut learning by combining quantitative performance metrics with visual explanation methods.

### Objectives

1.Analyse how self-supervised representation learning models (CLIP) build internal representations from image data.
2.Determine whether the model relies on semantically meaningful features or on irrelevant cues such as logos, text, or background artefacts.
3.Apply and compare Explainable AI techniques (Grad-CAM and LRP) to visualise and interpret model decisions.
4.Demonstrate the practical importance of interpretability for building reliable and trustworthy AI systems.
5.Quantify the extent of spurious-feature dependency across the ImageNet truck dataset.

## 2. Use Case Diagram & Descriptions



## 3. Functional Requirements

FR-1 Dataset Handling

- The system shall load image datasets from disk, torchvision, or Hugging Face and apply standard preprocessing (resize to 224×224, normalise per CLIP spec). [UC6, UC7]
- The system shall support at least the eight ImageNet truck classes defined in Section 6. [UC1]

FR-2 Model Integration

- The system shall load the pre-trained CLIP model (ViT-B/32 as default; RN50 as fallback). [UC1, UC5]
- The system shall perform zero-shot inference using configurable text prompts. [UC1, UC4]
- The system shall optionally load SimCLR or Barlow Twins for comparison. [UC5]

FR-3 Model Evaluation

- The system shall compute accuracy, precision, recall, and F1-score for both original and modified test images. [UC7]
- Results shall be stored in a structured format (CSV) for reproducibility. [UC8]

FR-4 Explainable AI

- The system shall implement Grad-CAM (primary) and LRP (optional) to generate attention heatmaps. [UC2]
- The system shall overlay heatmaps on original images and save visualisations as PNG. [UC2, UC4]

FR-5 Result Analysis

- The system shall calculate the Attention Alignment Score for each image. [UC1, UC3]
- The system shall flag images where >30 % of attention falls on annotated spurious regions. [UC3]
- The system shall compute the Modified Image Performance Delta (original accuracy − modified accuracy). [UC1]

FR-6 Visualisation & Dashboard

- The system shall provide an interactive web dashboard (Plotly Dash) with image upload, model selection, and real-time heatmap display. [UC4]
- The dashboard shall display predicted class, confidence score, and a plain-language attention summary. [UC4]
- The dashboard shall be deployed and accessible via a public Vercel URL. [UC4]

FR-7 Multi-Method Comparison

- The system shall generate Grad-CAM and LRP heatmaps for the same image set and compute their spatial correlation. [UC2, UC5]

FR-8 Experiment Management

- All experiments shall be reproducible via fixed random seeds and versioned code on GitHub. [UC8]
- The system shall support saving and reloading experiment configurations. [UC8]

## 4. Non-Functional Requirements

NFR-1 Performance

- Inference time per image: < 2 seconds on GPU.
- Heatmap generation per image: < 5 seconds.
- End-to-end explanation (inference + heatmap + display): < 8 seconds.
- Dashboard initial load: < 3 seconds.
- GPU memory usage: ≤ 8 GB (compatible with free Google Colab tier).

NFR-2 Security

- Uploaded images shall not be persisted beyond the active session unless explicitly saved by the user.

NFR-3 Usability

- The dashboard UI shall require no ML expertise to operate.
- Heatmap overlays shall use a perceptually uniform colour map (e.g., jet or viridis) with a legend.
- Plain-language attention summaries shall be shown alongside every heatmap.

NFR-4 Reliability

- The system shall checkpoint experiment results after every batch to prevent data loss (mitigates Colab disconnections).
- Results shall be deterministic given the same random seed and input data.

NFR-5 Scalability

- The architecture shall be modular to permit addition of new models (e.g., DINOv2, MAE) and new XAI methods without major refactoring.

NFR-6 Maintainability

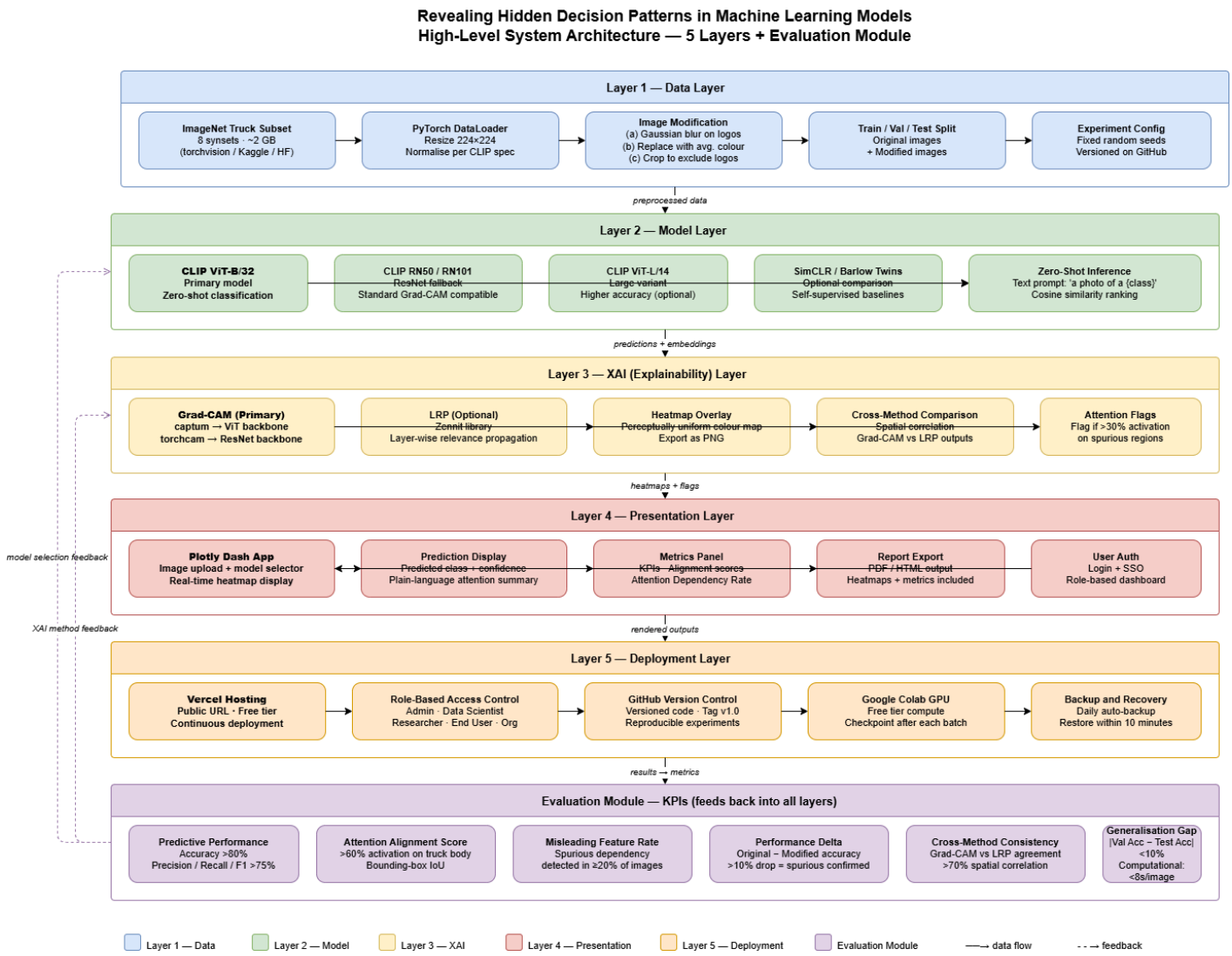
- Code shall be organised in clearly separated modules: data/, models/, xai/, dashboard/, evaluation/.
- All public functions shall include docstrings; the README shall document setup and reproduction steps.

NFR-7 Robustness

- The system shall handle corrupted or unusually shaped images with a graceful error message rather than a crash.
- Heatmap generation shall remain stable across CLIP ViT and ResNet backbones.

# 5. Software Architecture

## 5.1 Architecture Diagram



## 6. Data Persistence Strategy

### 6.1 Why No Database (or: Database Rationale)

**Why This Project Requires No Traditional Database**

This project is a research-oriented machine learning pipeline that analyses in unsupervised models applied to the ImageNet truck dataset. Unlike enterprise or web-based systems, it does not manage users, sessions, transactions, or real-time queries. All data artefacts are structured as flat files governed by a well-defined CSV schema, which is sufficient for the scope, scale, and reproducibility requirements of the project.

| Factor | Justification |
|---|---|
| Data volume & structure | The dataset consists of 10,659 labelled images (10,259 train + 400 test) spread across 8 truck classes. Metadata — file paths, class labels, split assignments, shortcut flags, and experiment results — is naturally tabular and small in volume. Three CSV files (train.csv, val.csv, test.csv) and one experiment log provide complete coverage. Relational overhead would add complexity without benefit. |
| Access patterns | All data access follows a simple sequential read pattern: load a CSV once at the start of a notebook, iterate over rows, produce outputs. There are no concurrent readers, no ad-hoc queries, no joins across large tables, and no write conflicts. SQLite or PostgreSQL would introduce an unnecessary query layer over operations that numpy and pandas already handle optimally. |
| Reproducibility & portability | CSV files checked into GitHub guarantee exact reproducibility across machines and environments (local, Colab, HuggingFace Spaces). A database server would require setup scripts, credentials, and network access that break Colab's stateless execution model and complicate peer review and grading. |

| | |
|---|---|
| Research conventions | The original paper (Kauffmann et al., Nature Machine Intelligence 2025) and its reference implementation use flat files throughout. Following the same convention ensures direct comparability of results and simplifies validation against published benchmarks. |
| Toolchain alignment | The full pipeline — PyTorch DataLoader, pandas, sklearn, Grad-CAM — is optimised for NumPy arrays and file-path strings. Introducing ORM mappings or SQL queries would add an impedance mismatch with no performance gain for this batch-processing workload. |

## 6.2 Data Storage Schema

**Folder Structure**

```
revealing_hidden_patterns/
├── data/
│   ├── raw/
│   │   ├── imagenet_trucks/
│   │   ├── chest_xray/
│   │   └── mvtec_anomaly/
│   ├── processed/
│   │   ├── train.csv
│   │   ├── val.csv
│   │   └── test.csv
│   ├── metadata/
│   └── dataset_registry.csv
├── models/
│   ├── pretrained/
│   └── checkpoints/
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_feature_extraction.ipynb
│   ├── 03_xai_gradcam.ipynb
│   └── 04_xai_lrp.ipynb
├── src/
│   ├── __init__.py
│   ├── dataset.py
│   ├── model.py
│   ├── gradcam.py
│   ├── lrp.py
│   └── evaluate.py
├── results/
│   ├── heatmaps/
│   └── experiment_log.csv
├── requirements.txt
└── README.md
```

**dataset_registry.csv**

| Field | Type | Description |
|---|---|---|
| dataset_id **PK** | string | Unique key e.g. imagenet_trucks |
| name | string | Human-readable dataset name |
| task_type | enum | classification / anomaly / medical |
| num_classes | int | Number of output classes |
| num_samples | int | Total images in dataset |
| image_size | string | Input resolution e.g. 224x224 |
| split_path | path | Relative path to processed/ |
| source_url | url | Original download link |

**train.csv / val.csv / test.csv**

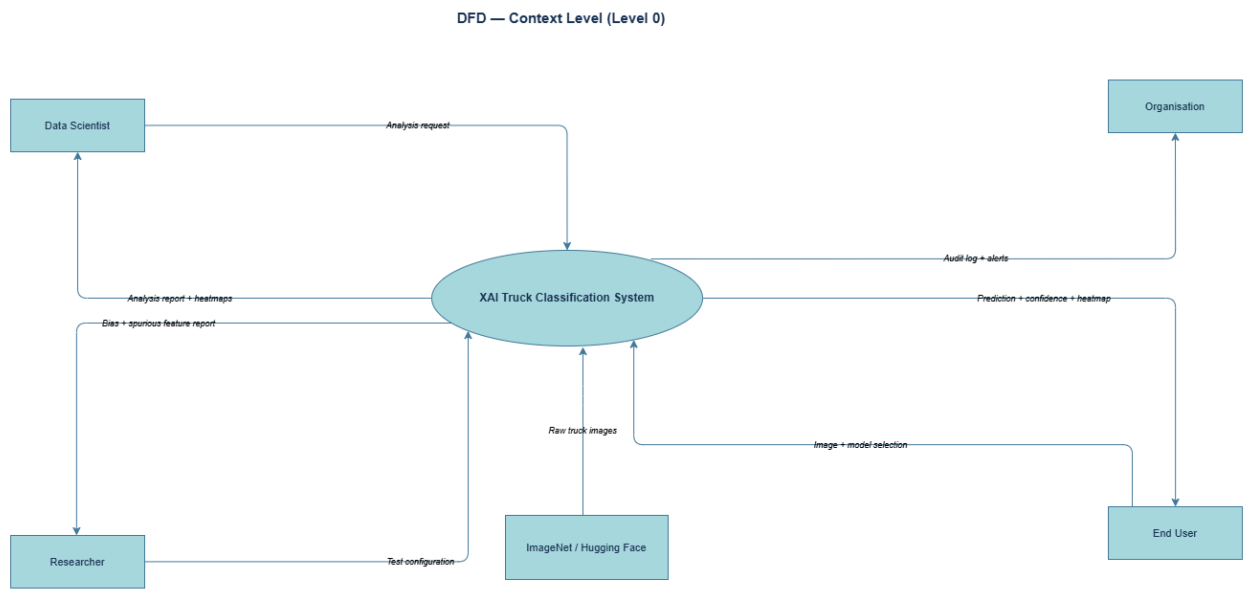
| Field | Type | Description |
|---|---|---|
| sample_id **PK** | string | Unique image ID |
| image_path | path | Relative path to image file |
| dataset_id **FK** | string | FK → dataset_registry |
| label | string | Ground-truth class name |
| label_id | int | Numeric class index |
| split | enum | train / val / test |
| has_shortcut | bool | Known Clever Hans artefact |
| notes | string | Optional annotation |

**experiment_log.csv**

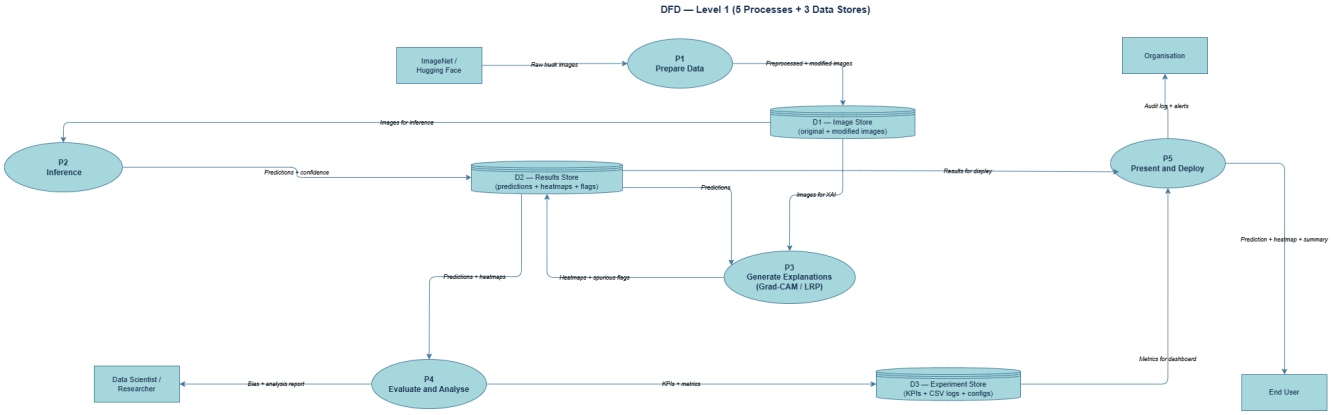
| Field | Type | Description |
|---|---|---|
| exp_id<br>PK | string | e.g. exp_20250429_143200 |
| model_name | enum | CLIP / SimCLR / BarlowTwins |
| dataset_id<br>FK | string | FK → dataset_registry |
| xai_method | enum | GradCAM / LRP / None |
| accuracy | float | Top-1 accuracy on test set |
| focus_score | float | XAI object-focus ratio 0-1 |
| shortcut_detected | bool | CH artefact found by XAI |
| heatmap_path | path | Path to results/heatmaps/ |
| timestamp | datetime | ISO 8601 run timestamp |
| notes | string | Observations / comments |

# 7. Data Flow Diagrams

## 7.1 Context-Level DFD (Level 0)

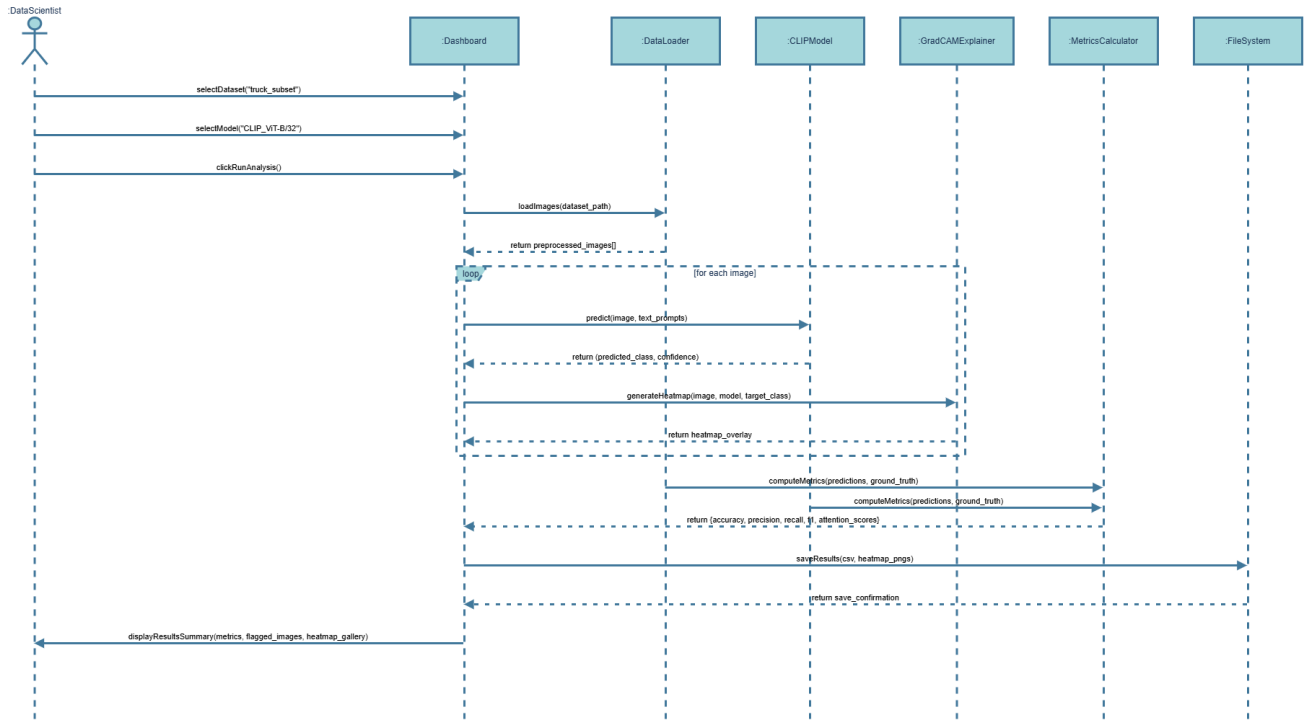


## 7.2 Level 1 DFD

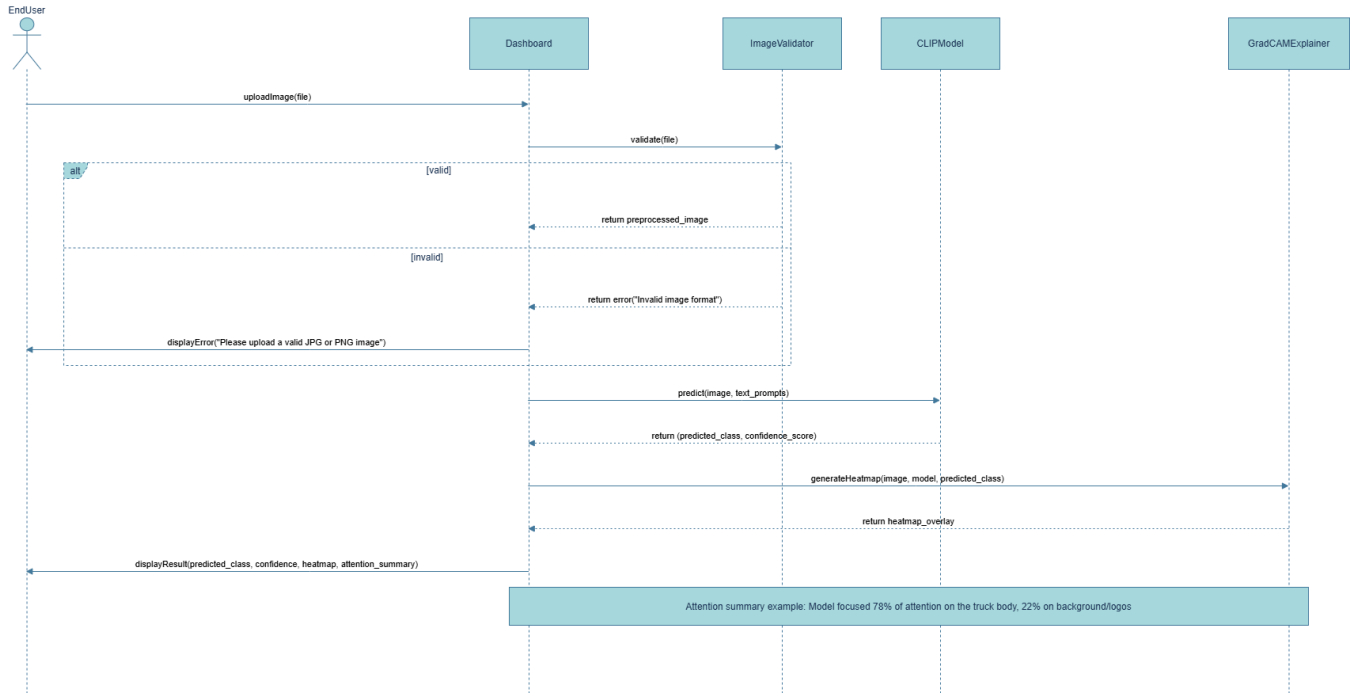


# 8. Sequence Diagrams

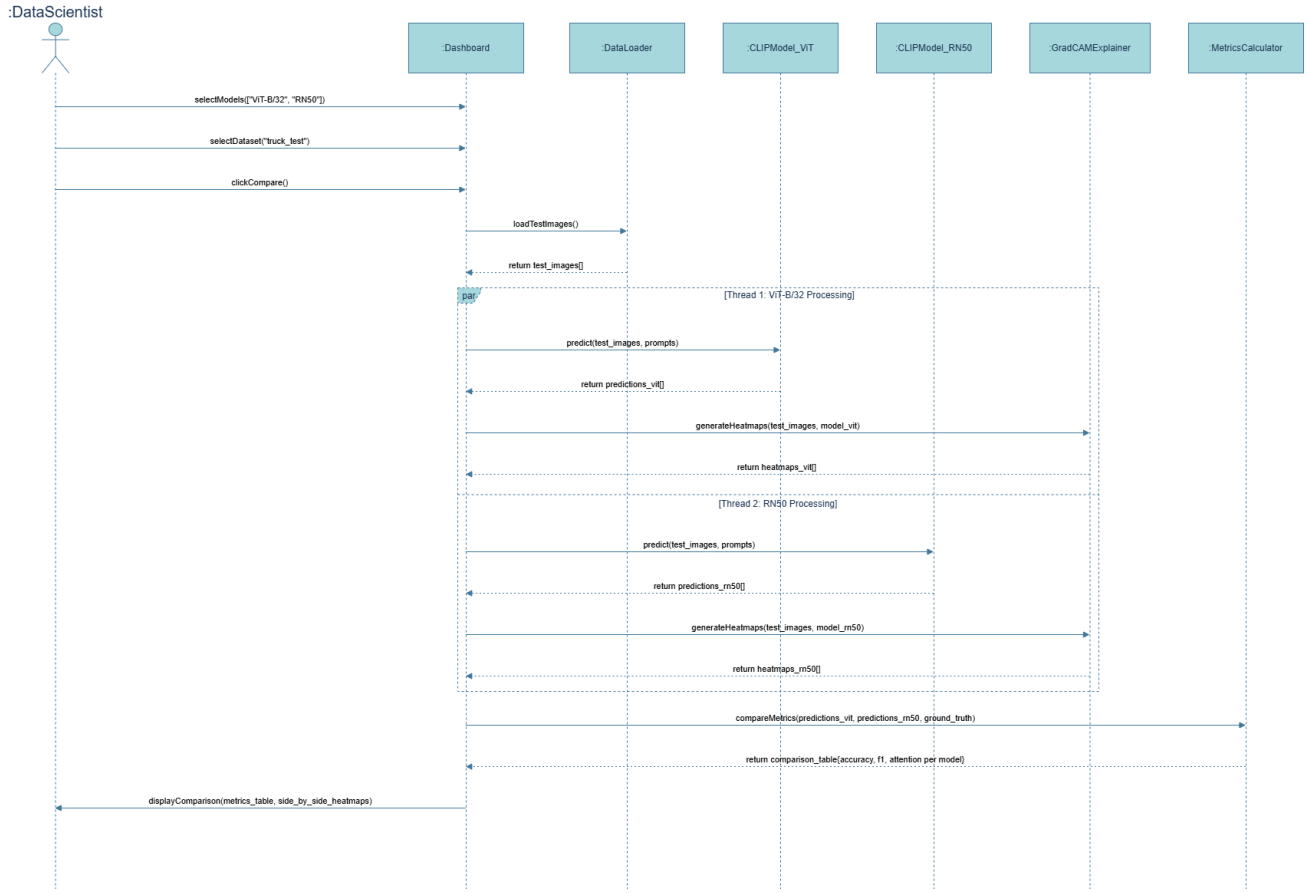
## 8.1 Sequence Diagram — UC1



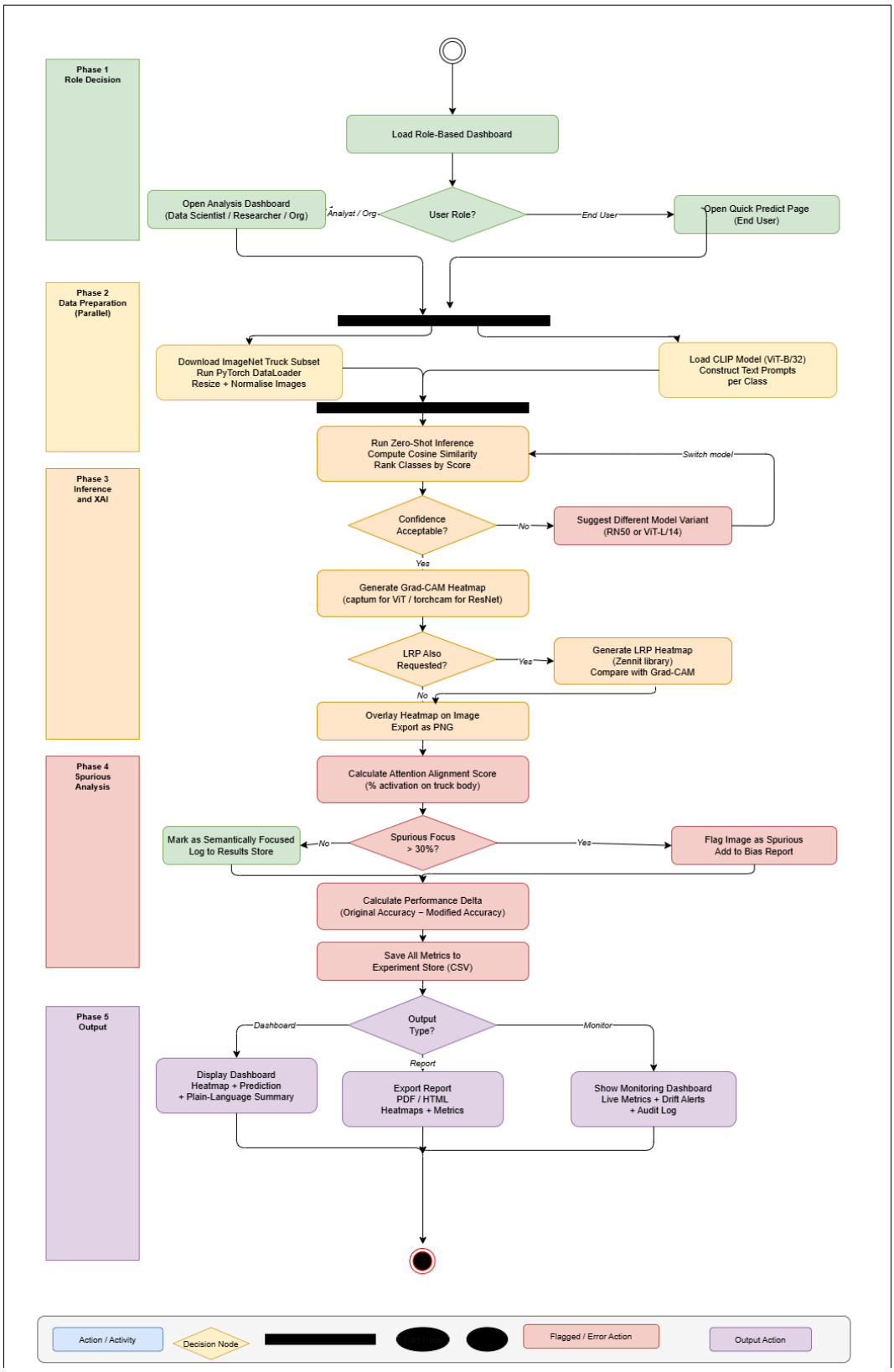

## 8.2 Sequence Diagram — UC4



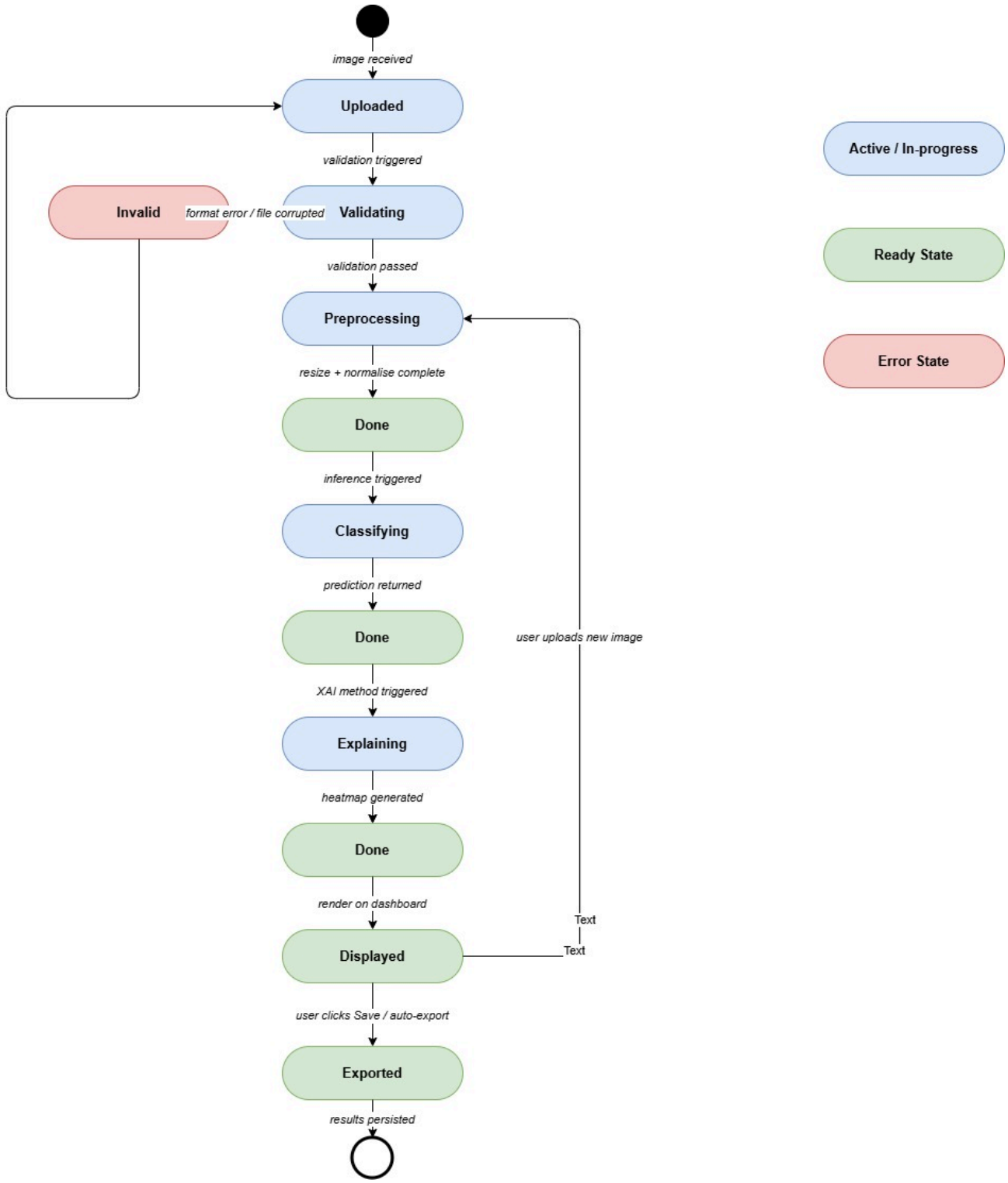
## 8.3 Sequence Diagram — UC5



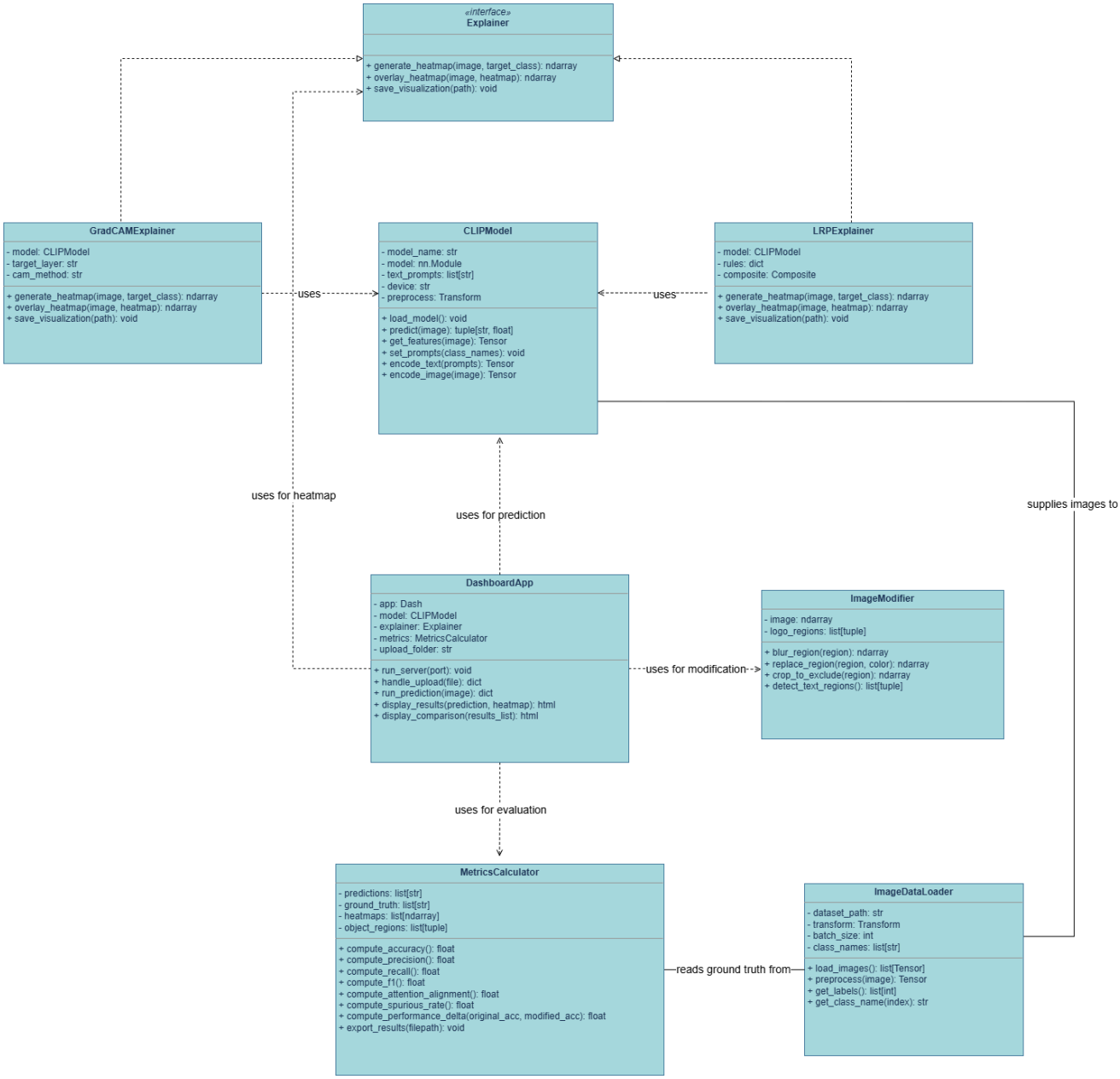
# 9. Activity Diagram



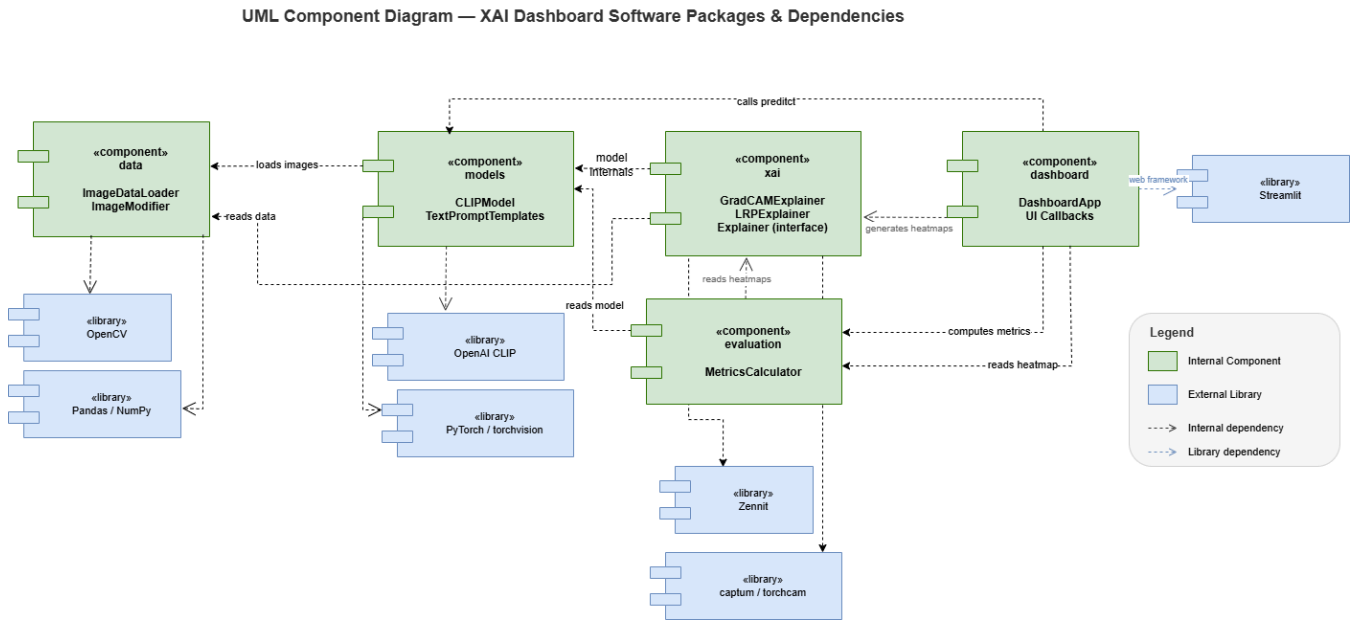
## 10. State Diagram



# 11. Class Diagram



# 12. Component Diagram



## 13. Deployment Diagram

The diagram below maps the five infrastructure nodes and describes the artefacts deployed to each, the communication protocols between them, and the direction of data flow during development, execution, and presentation.

| GitHub | Colab | HuggingFace | Vercel | Browser |
|---|---|---|---|---|
| Source of truth | Training & XAI execution | Model & demo hosting | Interactive dashboard | End user / evaluator |
| src/, notebooks/, data/processed/*.csv, requirements.txt, README.md | GPU runtime, pip install, feature extraction, Grad-CAM heatmaps, experiment_log.csv | Pretrained weights: CLIP, SimCLR, BarlowTwins, Gradio demo app | Next.js dashboard, results/heatmaps/, experiment_log.csv visualisation | Accesses Vercel dashboard and HuggingFace Spaces demo via HTTPS |
| git clone / git push | git clone via HTTPS | git push + Spaces API | GitHub Actions CI/CD | HTTPS / WebSocket |

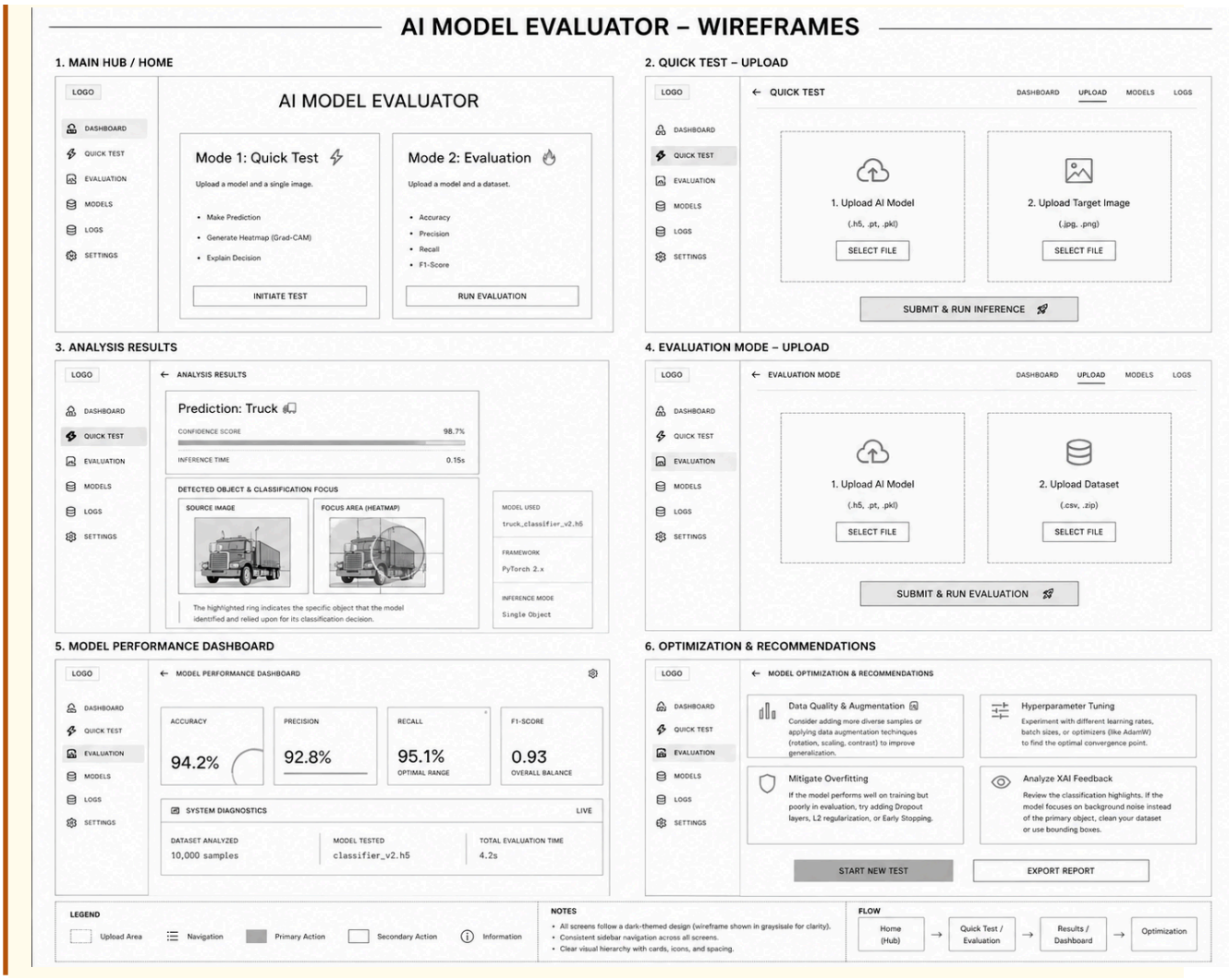
### Data Flow Summary

1. Developer pushes code and CSV splits to GitHub.
2. Colab clones the repository, installs dependencies, downloads pretrained weights from HuggingFace, runs feature extraction and XAI notebooks, and writes heatmaps and experiment_log.csv
back to the repo via git push.
3. A GitHub Actions workflow triggers a Vercel deployment whenever new results are pushed, rebuilding the Next.js results dashboard.
4. HuggingFace Spaces serves the interactive Gradio demo (model inference + Grad-CAM overlay) accessible from the Browser.
5. The evaluator opens both the Vercel dashboard and the HuggingFace demo in the Browser to inspect results without any local installation.

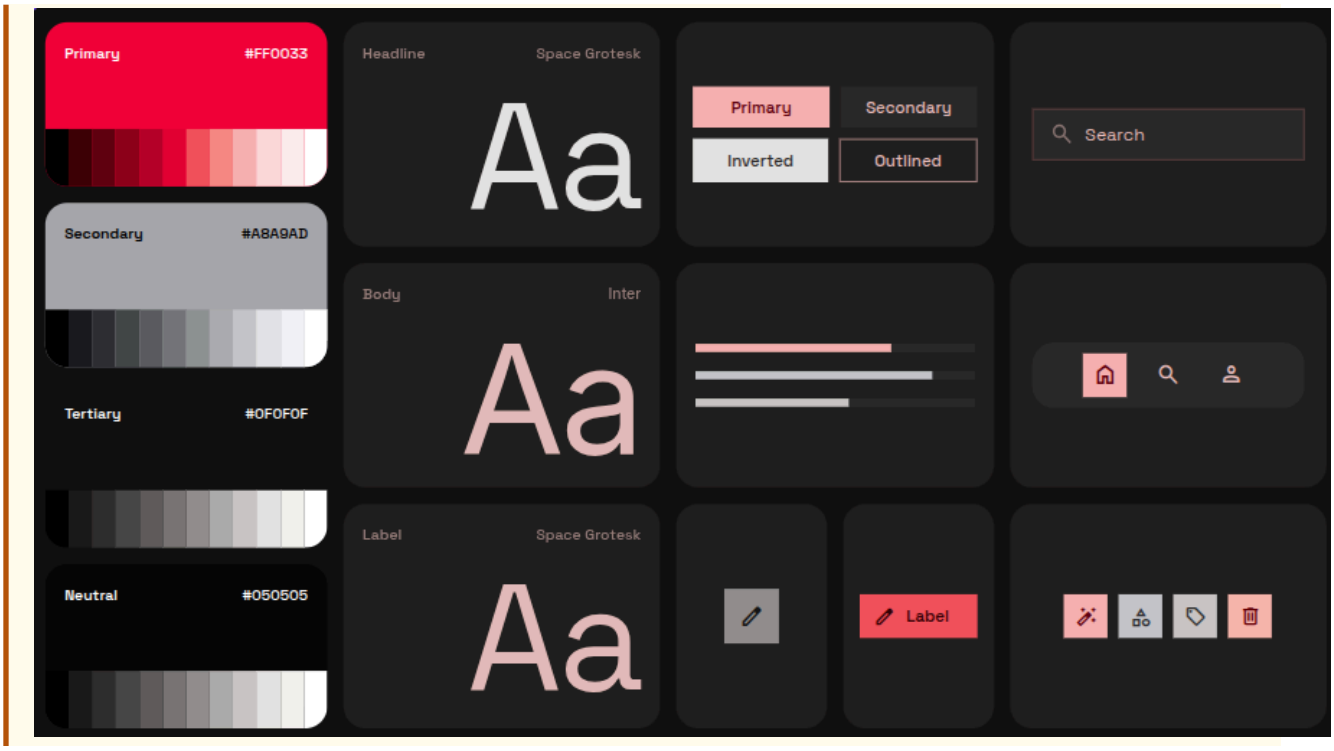
# 14. UI/UX Design

## 14.1 Wireframes

These wireframes represent the low-fidelity structure of the system interface, focusing on layout, user flow, and interaction elements without visual styling.

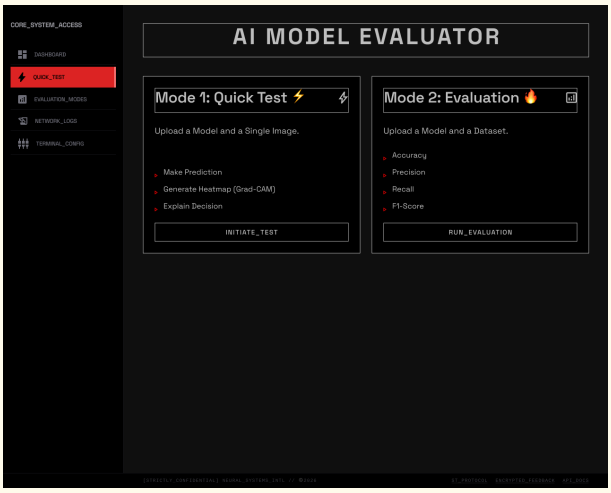


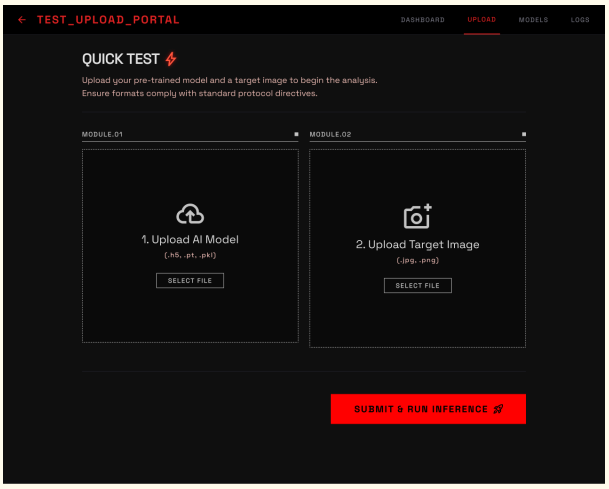
## 14.2 UI/UX Guidelines

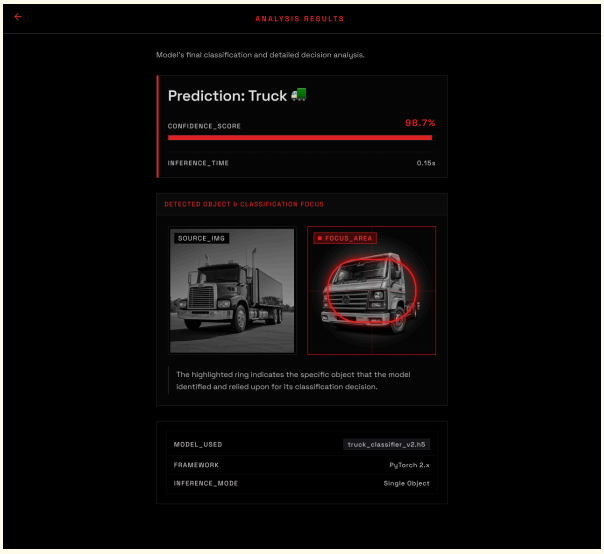


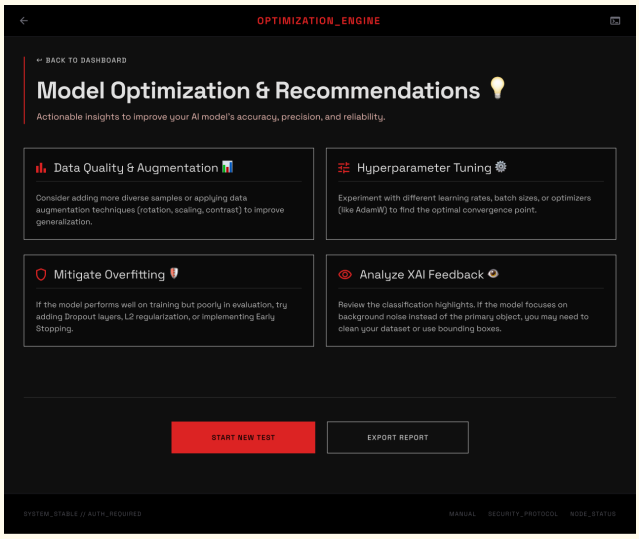
## 14.3 Figma

*Figma*



## 15. Technology Stack

The table below lists every technology used in the project, including version constraints, purpose, and licence.

| Layer | Technology | Version | Purpose | License |
|---|---|---|---|---|
| Language | Python | 3.9+ | Core programming language | PSF |
| Deep Learning | PyTorch | 2.0+ | Tensor operations, model loading, GPU compute | BSD-3 |
| Dataset | torchvision | 0.15+ | ImageNet dataset loading, image transforms | BSD-3 |
| Pre-trained Model | OpenAI CLIP | Latest | Zero-shot image classification via contrastive learning | MIT |
| XAI (ViT) | captum | 0.6+ | Grad-CAM for Vision Transformer architectures | BSD-3 |
| XAI (ResNet) | torchcam | 0.4+ | Grad-CAM for ResNet / CNN architectures | MIT |
| XAI (optional) | Zennit | 0.5+ | Layer-wise Relevance Propagation | MIT |
| Image Processing | OpenCV | 4.8+ | Image blurring, region modification, overlay compositing | Apache 2.0 |
| Data Analysis | Pandas | 2.0+ | CSV reading/writing, tabular result manipulation | BSD-3 |
| Numerical | NumPy | 1.24+ | Array operations, heatmap processing | BSD-3 |
| Visualisation | Matplotlib | 3.7+ | Heatmap colour mapping, chart generation | PSF |
| Dashboard | Plotly Dash | 2.14+ | Interactive web application framework | MIT |
| Deployment | Vercel | Free tier | Dashboard hosting with GitHub auto-deploy | Proprietary (free) |
| GPU Compute | Google Colab | Free tier | GPU-accelerated model inference and XAI generation | Proprietary (free) |
| Version Control | Git + GitHub | Latest | Source code versioning, result archiving, collaboration | Free |

## 16. Deployment Strategy

This section describes how the project is developed, versioned, deployed, and could be scaled beyond its current research scope.

### Development Environment

All model inference and XAI experiments run in Google Colab notebooks using the free T4 GPU tier. This provides sufficient compute for CLIP inference and Grad-CAM generation on the

ImageNet truck subset (~2 GB of images). Checkpoints are saved to Google Drive after each batch of images to prevent data loss from Colab session timeouts (sessions disconnect after approximately 90 minutes of inactivity or 12 hours maximum). The team uses a shared Colab notebook linked to the GitHub repository for consistency.

### Version Control

All source code, Jupyter notebooks, CSV result files, and generated heatmap PNGs are stored in a GitHub repository. The team follows a simple branching strategy: each pair works on a feature branch (e.g., pair-a/model-pipeline, pair-b/xai-integration, pair-c/dashboard) and merges to main via pull request after peer review. Milestones are tagged with semantic versions (e.g., v0.1-data-ready, v0.5-xai-working, v1.0-final). The README documents setup and reproduction steps.

### Dashboard Deployment

The Plotly Dash web application is deployed on Vercel's free tier. Vercel connects directly to the GitHub repository and auto-deploys the latest version of the main branch on every push. No CI/CD pipeline is required at the project's current scale. The free tier provides sufficient bandwidth for demonstration purposes and team access. The dashboard URL is shared with stakeholders for review and is included in the final submission.

### Scaling Considerations

The current architecture is designed for a research context: single-user analysis of hundreds of images. For production scaling, several changes would be needed: (1) model inference would move to a dedicated GPU server or cloud function (e.g., AWS Lambda with GPU), (2) the dashboard would be containerised with Docker for portable deployment, (3) result storage would migrate from CSV files to a PostgreSQL database, and (4) image storage would move to an object storage service such as AWS S3.

## 17. Internal API Documentation

This section documents the six core Python function interfaces that define the contracts between modules. Each entry specifies the full signature, parameter types, return values, usage example, and any exceptions raised.

### CLIPModel.predict

```
CLIPModel.predict(image: Tensor, prompts: list[str]) → tuple[str, float]
```

#### Description

Runs zero-shot classification on a single preprocessed image by computing cosine similarity between the image embedding and all text prompt embeddings. Returns the class with the highest similarity.

#### Parameters

| Name | Type | Required | Description |
|---|---|---|---|
| image | `torch.Tensor` | Yes | Preprocessed image tensor, shape [3, 224, 224], normalised per CLIP spec |
| prompts | `list[str]` | Yes | Text prompts for each class, e.g., ["a photo of a beer truck", "a photo of a fire engine", ...] |

#### Returns

| Name | Type | Description |
|---|---|---|
| predicted_class | `str` | Class name with highest cosine similarity |
| confidence | `float` | Cosine similarity score (range 0.0 to 1.0) |

#### Example

```
model = CLIPModel("ViT-B/32")
model.load_model()
pred_class, conf = model.predict(image_tensor, truck_prompts)
# pred_class = "beer_truck", conf = 0.87
```

**Raises:** `ValueError if prompts list is empty. RuntimeError if model not loaded.`

## GradCAMExplainer.generate_heatmap

```
GradCAMExplainer.generate_heatmap(image: Tensor, target_class: str) → ndarray
```

### Description

Generates a Grad-CAM attention heatmap for the given image and target class. Computes gradients of the target class score with respect to feature maps of the target layer, then produces a 2D attention map.

### Parameters

| Name | Type | Required | Description |
|---|---|---|---|
| image | `torch.Tensor` | Yes | Preprocessed image tensor, shape [3, 224, 224] |
| target_class | `str` | Yes | Class name to explain (typically the predicted class) |

### Returns

| Name | Type | Description |
|---|---|---|
| heatmap | `numpy.ndarray` | 2D attention map, shape [224, 224], values normalised to [0, 1] |

### Example

```
explainer = GradCAMExplainer(model, target_layer="layer4")
heatmap = explainer.generate_heatmap(image_tensor, "beer_truck")
# heatmap.shape = (224, 224), heatmap.max() = 1.0
```

---

## GradCAMExplainer.overlay_heatmap

```
GradCAMExplainer.overlay_heatmap(image: ndarray, heatmap: ndarray, alpha: float = 0.6) → ndarray
```

### Description

Overlays the Grad-CAM heatmap on the original image using a colour map. Returns a composite image ready for display or saving.

### Parameters

| Name | Type | Required | Default | Description |
|---|---|---|---|---|
| image | `numpy.ndarray` | Yes | — | Original image, shape [H, W, 3], uint8 |
| heatmap | `numpy.ndarray` | Yes | — | Attention map, shape [H, W], float [0, 1] |
| alpha | `float` | No | 0.6 | Opacity of heatmap overlay (0 = invisible, 1 = fully opaque) |

### Returns

| Name | Type | Description |
|---|---|---|
| overlay | `numpy.ndarray` | Composite image, shape [H, W, 3], uint8 |

### Example

```
overlay = explainer.overlay_heatmap(original_image, heatmap, alpha=0.6)
cv2.imwrite("heatmap_overlay.png", overlay)
```

---

## MetricsCalculator.compute_metrics

```
MetricsCalculator.compute_metrics(predictions: list[str], ground_truth:
list[str]) → dict
```

### Description

Computes classification metrics by comparing predictions against ground truth labels. Returns a dictionary with overall and per-class metrics.

### Parameters

| Name | Type | Required | Description |
|---|---|---|---|
| predictions | `list[str]` | Yes | Predicted class names for each test image |
| ground_truth | `list[str]` | Yes | True class names for each test image |

### Returns

| Name | Type | Description |
|---|---|---|
| metrics | `dict` | Dictionary containing: accuracy (float), precision (float), recall (float), f1_score (float), per_class (dict of class-level metrics) |

### Example

```
calc = MetricsCalculator()
results = calc.compute_metrics(pred_list, true_list)
# results = {"accuracy": 0.835, "precision": 0.81, "recall": 0.79,
#            "f1_score": 0.80, "per_class": {...}}
```

---

## ImageDataLoader.load_images

```
ImageDataLoader.load_images(dataset_path: str, split: str = "test") → tuple[list[Tensor], list[str]]
```

### Description

Loads all images from the specified dataset directory, applies CLIP preprocessing transforms, and returns preprocessed tensors with their corresponding class labels.

### Parameters

| Name | Type | Required | Default | Description |
|---|---|---|---|---|
| dataset_path | `str` | Yes | — | Path to dataset root directory |
| split | `str` | No | "test" | Which split to load: "train", "val", or "test" |

### Returns

| Name | Type | Description |
|---|---|---|
| images | `list[Tensor]` | List of preprocessed image tensors, each shape [3, 224, 224] |
| labels | `list[str]` | Corresponding class name for each image |

### Example

```
loader = ImageDataLoader()
images, labels = loader.load_images("data/raw/", split="test")
# len(images) = 100, images[0].shape = torch.Size([3, 224, 224])
```

---

## ImageModifier.blur_region

```
ImageModifier.blur_region(image: ndarray, region: tuple[int, int, int, int], kernel_size: int = 51) → ndarray
```

### Description

Applies Gaussian blur to a specific rectangular region of the image (typically a logo or text area) while leaving the rest of the image unchanged. Used for the Logo/Text Dependency Test.

### Parameters

| Name | Type | Required | Default | Description |
|---|---|---|---|---|
| image | `numpy.ndarray` | Yes | — | Original image, shape [H, W, 3], uint8 |
| region | `tuple[int,int,int,int]` | Yes | — | Bounding box (x1, y1, x2, y2) of the region to blur |
| kernel_size | `int` | No | 51 | Gaussian blur kernel size (must be odd) |

### Returns

| Name | Type | Description |
|---|---|---|
| modified_image | `numpy.ndarray` | Image with the specified region blurred, shape [H, W, 3] |

### Example

```
modifier = ImageModifier()
blurred = modifier.blur_region(truck_image, region=(100, 50, 300, 150),
kernel_size=51)
cv2.imwrite("blurred_logo.png", blurred)
```

## 18. Testing & Validation Plan

## Section 1 — Unit Tests

Unit tests verify that each individual component functions correctly in isolation, independent of other modules. All tests should be automated using pytest where feasible.

| Test ID | Component | Test Description | Input | Expected Result | Priority |
|---|---|---|---|---|---|
| **UT-1** | ImageDataLoader | Load 1 image from disk; verify output tensor shape | 1 JPEG image | Tensor shape [3, 224, 224] | **High** |
| **UT-2** | ImageDataLoader | Load image with invalid format (PDF); verify graceful failure | 1 PDF file | Graceful error message displayed; no crash or unhandled exception | **Medium** |
| **UT-3** | CLIPModel | Run predict() on 1 preprocessed image; verify output types | Tensor [3, 224, 224] | Returns tuple (string, float) where float is in range [0, 1] | **High** |
| **UT-4** | CLIPModel | Run predict() with empty text prompts list; verify graceful failure | 1 tensor + empty prompt list | Graceful error message; no crash | **Medium** |
| **UT-5** | GradCAMExplainer | Generate heatmap for 1 image; verify output array dimensions | 1 image + model + class index | ndarray shape matches input image spatial dimensions | **High** |
| **UT-6** | GradCAMExplainer | Test heatmap generation with both ViT-B/32 and RN50 backbones | 1 image per backbone | Both produce valid heatmap ndarrays without errors or shape mismatch | **High** |
| **UT-7** | MetricsCalculator | Pass known predictions + ground truth; verify accuracy calculation | 10 predictions (7 correct, 3 wrong) | Returns accuracy = 0.70; matches manual calculation | **High** |
| **UT-8** | ImageModifier | Apply Gaussian blur to a defined region; verify only target pixels changed | 1 image + bounding box coordinates | Pixels inside region modified; pixels outside region remain unchanged | **Medium** |

## Section 2 — Integration Tests

Integration tests validate that the components developed by Pairs A, B, and C work correctly together as an end-to-end pipeline. These tests are run after all unit tests pass and before dashboard deployment.

| Test ID | Components | Test Description | Expected Result | Priority |
|---|---|---|---|---|
| **IT-1** | DataLoader → CLIPModel | Load a batch of 10 images via DataLoader; run CLIP zero-shot inference on all 10 | 10 valid prediction tuples (class string, confidence float) returned; no errors | **High** |

| Test ID | Components | Test Description | Expected Result | Priority |
|---|---|---|---|---|
| **IT-2** | CLIPModel → GradCAMExplainer | Run predict() on a single image; pass predicted class to GradCAMExplainer | Heatmap generated successfully; ndarray shape matches input image dimensions | **High** |
| **IT-3** | ImageModifier → CLIPModel | Blur logo regions on 20 images using ImageModifier; run CLIP inference on all 20 | Predictions returned for all 20 modified images; no shape or inference errors | **High** |
| **IT-4** | Full pipeline → FileSystem | Run complete analysis pipeline on 10 images; verify all output files are saved | predictions.csv exists with columns: filename, predicted_class, confidence, alignment_score. PNG heatmaps exist for all 10 images | **High** |
| **IT-5** | Full pipeline → Dashboard | Upload one image via the dashboard UI; trigger inference and explanation | Prediction and heatmap displayed within 8 seconds; attention summary text visible on page | **Medium** |

## Section 3 — User Acceptance Testing (UAT)

UAT is conducted from the perspective of each stakeholder role (Data Scientist, End User, Organisation) using the deployed Vercel dashboard. Tests confirm that real-world usage scenarios meet the acceptance criteria defined in the functional requirements.

| Test ID | Scenario | Steps | Acceptance Criteria | Priority |
|---|---|---|---|---|
| **UAT-1** | Data scientist runs full batch analysis | 1. Select dataset. 2. Select CLIP model. 3. Click Run Analysis. 4. Wait for completion. | All per-class and overall metrics displayed (accuracy, precision, recall, F1). All heatmaps generated. Results downloadable as CSV via dashboard. | **High** |
| **UAT-2** | End user uploads a single image | 1. Navigate to Quick Predict page. 2. Upload a valid truck JPEG. 3. Click Explain. | Predicted class and heatmap overlay displayed within 8 seconds. Plain-language attention summary shown (e.g., 'Model focused 78% of attention on truck body'). | **High** |
| **UAT-3** | User uploads an invalid file type | 1. Navigate to Quick Predict page. 2. Upload a PDF file. 3. Click Explain. | Friendly error message displayed: 'Please upload a valid JPG or PNG image.' No crash; page remains functional. | **Medium** |
| **UAT-4** | User compares two CLIP model variants | 1. Open Model Comparison tool. 2. Select ViT-B/32 and RN50. 3. Click Compare. | Side-by-side accuracy and F1 metrics table rendered correctly. Side-by-side heatmap comparisons displayed for at least 3 sample images. | **Medium** |
| **UAT-5** | User accesses dashboard via Vercel URL | 1. Open Vercel deployment URL in Chrome. 2. Reload in Firefox. | Page loads in under 3 seconds in both browsers. All buttons, dropdowns, image upload control, and heatmap display render correctly with no console errors. | **High** |